

# Anotações — Aula 2: Pré-processamento de Dados

**Disciplina:** Machine Learning — Pós-Graduação Ciência de Dados e Big Data / Estatística

**Professor:** Murilo Afonso Robiati Bigoto (PUC Minas)

**Dataset usado:** dados\_cancelamento.csv — 2000 clientes, 19 colunas, target: churn

*Frase-chave da aula: "Dados são o novo petróleo, mas o Feature Engineering é a refinaria."*

## Recap da Aula 1

- IA > Machine Learning > Deep Learning (do maior ao mais específico)
- Tipos de aprendizado: **supervisionado, não-supervisionado, por reforço**
- **Overfitting** = modelo decora os dados de treino, vai mal em dados novos
- **Underfitting** = modelo simples demais, não aprende nem o treino
- Identificação prática: comparar performance no conjunto de **treino vs teste**

## Pipeline Completo de Pré-processamento (Visão Geral)

### Dataset Bruto

#### 1. EDA (Análise Exploratória)

missing, outliers, assimetria, correlações

#### 2. Tratamento de Missing Values

Mediana    saldo\_conta, salario\_anual (assimétricos)

Média      score\_credito (simétrico)

KNN        num\_transacoes (relacional)

Moda       nivel\_educacao (categórico)

#### 3. Tratamento de Outliers

Capping (Winsorização)    percentis 1% / 99%  
Transformação log        reduzir assimetria extrema

#### 4. Encoding Categórico

Label Encoding    nivel\_educacao (ordinal)  
Binário        genero  
One-Hot        pais, tipo\_conta (nominais)

#### 5. Feature Engineering

saldo\_por\_produto, score\_fidelidade, faixa\_etaria, transacoes\_por\_produto

#### 6. Escalonamento (StandardScaler)

#### 7. Seleção de Features (Boruta)

#### 8. Dataset pronto para Modelagem

---

## Bibliotecas da Aula

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from scipy import stats

# Sklearn — pré-processamento
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneH
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Boruta (seleção de variáveis)
from boruta import BorutaPy

# Configurações visuais
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 110
```

```
pd.set_option('display.max_columns', 50)
np.random.seed(42)
```

## ETAPA 1 — Contexto e Leitura dos Dados

**Problema:** Um banco quer prever quais clientes têm mais chance de cancelar seus serviços (**churn**).

```
df_raw = pd.read_csv('dados_cancelamento.csv')

# Visão geral
print(f"Dimensões: {df_raw.shape[0]} linhas × {df_raw.shape[1]} colunas")
print(df_raw.dtypes)
print(df_raw.describe())
```

**Estrutura do dataset ( dados\_cancelamento.csv ):**

| Coluna         | Tipo  | Descrição                                  |
|----------------|-------|--------------------------------------------|
| id_cliente     | int   | Identificador único                        |
| idade          | int   | Idade do cliente                           |
| genero         | str   | Masculino / Feminino                       |
| pais           | str   | País de origem                             |
| nivel_educacao | str   | Ensino Médio / Graduação / Pós / Doutorado |
| score_credito  | float | Score de risco (tipo Serasa)               |
| saldo_conta    | float | Saldo bancário                             |
| salario_anual  | float | Salário anual declarado                    |
| num_produtos   | int   | Produtos contratados                       |
| num_transacoes | float | Transações realizadas                      |
| tempo_cliente  | float | Tempo de relacionamento                    |
| churn          | int   | <b>TARGET:</b> 0 = ficou, 1 = cancelou     |

ruido\_1/2/3

—

Variáveis irrelevantes (para treino do Boruta)

*Taxa de churn: ~12% — dataset com desbalanceamento moderado (88% não cancelou)*

## ETAPA 2 — Análise Exploratória (EDA)

### 2.1 — EDA Automatizada com ydata-profiling

```
from ydata_profiling import ProfileReport

profile = ProfileReport(
    df_raw,
    title="Relatório EDA — Churn Bancário",
    explorative=True,
    minimal=False
)

profile.to_file("relatorio_eda.html") # salva como arquivo
profile.to_notebook_iframe()         # exibe no Colab
```

*Analogia: O ydata-profiling é como uma radiografia do seu dataset. Em segundos você vê os “ossos quebrados” — missing, assimetrias, correlações perigosas — antes de começar o tratamento.*

#### O que observar no relatório:

- **Overview:** alertas automáticos (missing, alta cardinalidade, correlações altas)
- **Variables:** distribuição de cada variável
- **Correlations:** mapa de calor entre variáveis
- **Missing Values:** padrão de ausência — aleatório ou estruturado?

### 2.2 — EDA Manual: Correlações e Distribuições

```
# Mapa de valores missing por variável
miss_df = pd.DataFrame({
    'Missing (n)': df_raw.isnull().sum(),
    'Missing (%)': (df_raw.isnull().mean() * 100).round(2)
```

```
}).query('`Missing (n)` > 0').sort_values('Missing (%)', ascending=False)

print(miss_df)
# salario_anual: ~12% de missing — a variável com mais ausências
```

### Observações do professor sobre as variáveis:

- `salario_anual` : 12% de missing    perguntar: é MCAR, MAR ou MNAR? (pessoa não quis declarar?)
- `tempo_cliente` × `churn` : correlação inversa — quanto menor o tempo, maior a chance de cancelar
- `saldo_normalizado` × `saldo_conta` : alta correlação entre si (uma é a versão escalonada da outra)
- `num_transacoes` : mediana igual entre quem cancela e quem não cancela    pode não ser boa feature

## ETAPA 3 — Tratamento de Missing Values

### Tipos de Missing (MCAR / MAR / MNAR)

| Tipo        | Nome                         | O que significa                                 |
|-------------|------------------------------|-------------------------------------------------|
| <b>MCAR</b> | Missing Completely at Random | Ausência aleatória, sem padrão                  |
| <b>MAR</b>  | Missing at Random            | Ausência explicada por outra variável           |
| <b>MNAR</b> | Missing Not at Random        | Ausência intencional (ex: salário alto omitido) |

### Estratégias de Imputação

| Técnica            | Quando usar                                  |
|--------------------|----------------------------------------------|
| <b>Média</b>       | Dados simétricos, sem outliers extremos      |
| <b>Mediana</b>     | Dados assimétricos ou com outliers           |
| <b>KNN Imputer</b> | Quando há relação estrutural entre variáveis |
| <b>Moda</b>        | Variáveis categóricas                        |

***Analogia:** Imputar pela média numa variável assimétrica é como calcular a “renda média do Brasil” incluindo bilionários — o resultado não representa ninguém. A mediana é mais honesta.*

## Código — Imputação no Dataset de Churn

```
from sklearn.impute import SimpleImputer, KNNImputer

df = df_raw.copy() # sempre trabalhe numa cópia!

print(" Antes da imputação:")
print(df[['saldo_conta', 'score_credito', 'salario_anual', 'num_transacoes']].isnull().sum())

# 1. MEDIANA — variáveis assimétricas (saldo e salário)
median_imputer = SimpleImputer(strategy='median')
df[['saldo_conta', 'salario_anual']] = median_imputer.fit_transform(
    df[['saldo_conta', 'salario_anual']])

# 2. MÉDIA — score_credito (distribuição mais simétrica)
mean_imputer = SimpleImputer(strategy='mean')
df[['score_credito']] = mean_imputer.fit_transform(df[['score_credito']])
df['score_credito'] = df['score_credito'].round().astype(int)

# 3. KNN — num_transacoes (usa saldo e num_produtos como vizinhos)
knn_imputer = KNNImputer(n_neighbors=5)
df[['num_transacoes']] = knn_imputer.fit_transform(
    df[['num_transacoes', 'saldo_conta', 'num_produtos']][:, [0]])
df['num_transacoes'] = df['num_transacoes'].round().astype(int)

# 4. MODA — nivel_educacao (variável categórica)
moda = df['nivel_educacao'].mode()[0]
df['nivel_educacao'] = df['nivel_educacao'].fillna(moda)

print("\n Após a imputação:")
print(df[['saldo_conta', 'score_credito', 'salario_anual', 'num_transacoes', 'nivel_educacao']])
```

*Por que `[:, [0]]` no KNN? O `KNNImputer` recebe uma matriz com múltiplas colunas (para calcular os vizinhos), mas queremos apenas a coluna imputada de volta — o `[:, [0]]` seleciona somente a primeira coluna do resultado.*

## ETAPA 4 — Tratamento de Outliers

### O que são outliers?

*Analogia: Outlier é o aluno que tira 10 numa turma com média 5 — ou o que tira 0. Eles distorcem a média e podem enganar o modelo, fazendo-o aprender padrões que só existem naqueles casos extremos.*

### Métodos de detecção: IQR (Intervalo Interquartílico)

```
def detectar_outliers_iqr(series, nome):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((series < lower) | (series > upper)).sum()
    print(f" {nome}: {n_out} outliers | limites [{lower:,.0f}, {upper:,.0f}]")
    return lower, upper

print(" Detecção de Outliers (IQR):")
for col in ['saldo_conta', 'salario_anual', 'num_transacoes']:
    detectar_outliers_iqr(df[col], col)
```

### Estratégias de tratamento

| Método                    | O que faz                                                 |
|---------------------------|-----------------------------------------------------------|
| Trimming                  | Remove os outliers do dataset                             |
| Capping (Winsorização)    | Limita ao percentil 1% e 99% — não remove, apenas “apara” |
| Transformação logarítmica | Comprime a escala, reduz assimetria extrema               |

```
# CAPPING — limitar ao percentil 1% e 99%
for col in ['saldo_conta', 'salario_anual']:
    p01 = df[col].quantile(0.01)
```

```

p99 = df[col].quantile(0.99)
df[f'{col}_capped'] = df[col].clip(lower=p01, upper=p99)
print(f" {col}: máx ANTES = {df[col].max():.0f} | APÓS = {df[f'{col}_capped'].max():.0f}"

# TRANSFORMAÇÃO LOGARÍTMICA — reduz assimetria
df['log_saldo_conta'] = np.log1p(df['saldo_conta_capped'])
df['log_salario_anual'] = np.log1p(df['salario_anual_capped'])

```

Por que `np.log1p()` e não `np.log()` ? O `log1p(x)` calcula  $\log(1 + x)$ , o que evita erros com valores zero. É uma prática segura para variáveis financeiras que podem ter saldo = 0.

## ETAPA 5 — Encoding de Variáveis Categóricas

Modelos de ML trabalham com **números**, não com texto. Precisamos converter categorias em valores numéricos — mas a forma como fazemos isso importa.

| Tipo de variável           | Técnica                       | Exemplo                    |
|----------------------------|-------------------------------|----------------------------|
| <b>Nominal</b> (sem ordem) | One-Hot Encoding              | pais , tipo_conta          |
| <b>Ordinal</b> (com ordem) | Label Encoding com mapeamento | nivel_educacao             |
| <b>Binária</b>             | 0 / 1 direto                  | genero ,<br>cartao_credito |

### Label Encoding — variável ordinal

```

# Mapeamento manual respeitando a ordem
ordem_educacao = {'Ensino Médio': 0, 'Graduação': 1, 'Pós-Graduação': 2, 'Doutorado': 3}
df['nivel_educacao_enc'] = df['nivel_educacao'].map(ordem_educacao)

print(df[['nivel_educacao', 'nivel_educacao_enc']].drop_duplicates().sort_values('nivel_ed

```

Por que mapear manualmente? Se usarmos o `LabelEncoder` do `sklearn` sem ordem, ele pode codificar “Doutorado” = 0 e “Ensino Médio” = 3, invertendo a lógica. Para variáveis ordinais, sempre mapeie manualmente.

## Encoding binário

```
df['genero_enc'] = (df['genero'] == 'Feminino').astype(int)
# Feminino = 1, Masculino = 0
```

## One-Hot Encoding — variáveis nominais

```
df = pd.get_dummies(df, columns=['pais', 'tipo_conta'], drop_first=True, dtype=int)

ohe_cols = [c for c in df.columns if c.startswith('pais_') or c.startswith('tipo_conta_')]
print(f"One-Hot Encoding gerou {len(ohe_cols)} novas colunas: {ohe_cols}")
```

*O que é `drop_first=True` ? É para evitar a **Dummy Variable Trap** (multicolinearidade perfeita). Se você tem as colunas `pais_Brasil` , `pais_Argentina` e `pais_Chile` , a informação de qualquer uma delas já está contida nas outras — ter as 3 é redundante. `drop_first=True` remove uma delas automaticamente.*

---

## ETAPA 6 — Feature Engineering (Engenharia de Atributos)

Criar novas variáveis a partir das existentes usando **conhecimento de domínio**.

***Analogia:** Feature Engineering é como cozinhar. Você tem os ingredientes brutos (as variáveis), mas combiná-los corretamente — uma pitada daqui, uma proporção dali — é o que transforma uma receita mediana em um prato excelente.*

```
# Razão: saldo por produto (comportamento financeiro)
df['saldo_por_produto'] = df['log_saldo_conta'] / (df['num_produtos'] + 1)

# Interação: score × tempo (clientes fiéis e confiáveis)
df['score_fidelidade'] = df['score_credito'] * df['tempo_cliente']

# Variável derivada discreta: faixa etária
df['faixa_etaria'] = pd.cut(df['idade'],
                             bins=[17, 30, 45, 60, 100],
                             labels=['18-30', '31-45', '46-60', '61+'])
df['faixa_etaria_enc'] = df['faixa_etaria'].cat.codes
```

```
# Proporção: transações por produto
df['transacoes_por_produto'] = df['num_transacoes'] / (df['num_produtos'] + 1)

print("  Novas features criadas:")
new_features = ['saldo_por_produto', 'score_fidelidade', 'faixa_etaria_enc', 'transacoes_p
print(df[new_features].describe().T[['mean', 'std', 'min', 'max']])
```

Por que `+ 1` nos denominadores? Para evitar divisão por zero caso `num_produtos = 0`.

## ETAPA 7 — Escalonamento de Features

### Por que escalonar?

Algoritmos baseados em **distância** (KNN, SVM) ou **gradiente descendente** (Redes Neurais, Regressão Logística) são sensíveis à escala das variáveis.

***Analogia:** Imagine comparar o peso de uma pessoa (70 kg) com sua altura (1,75 m). Se não escalonarmos, o peso vai “dominar” o cálculo de distância só por ter números maiores — como se 1 kg valesse mais do que 1 cm. O escalonamento coloca tudo na mesma régua.*

| Técnica                         | Fórmula                      | Quando usar                   |
|---------------------------------|------------------------------|-------------------------------|
| <b>StandardScaler</b> (Z-Score) | $(x - \mu) / \sigma$         | Dados aproximadamente normais |
| <b>MinMaxScaler</b>             | $(x - \min) / (\max - \min)$ | Dados com limites conhecidos  |

```
from sklearn.preprocessing import StandardScaler

# Definir features para modelagem
feature_cols = [
    'idade', 'score_credito', 'log_saldo_conta', 'log_salario_anual',
    'num_produtos', 'num_transacoes', 'tempo_cliente',
    'cartao_credito', 'membro_ativo', 'genero_enc',
    'nivel_educacao_enc', 'faixa_etaria_enc',
    'saldo_por_produto', 'score_fidelidade', 'transacoes_por_produto',
```

```

    'ruído_1', 'ruído_2'
] + ohe_cols

X = df[feature_cols].copy()
y = df['churn'].copy()

# StandardScaler: média = 0, desvio padrão = 1
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

print(" Comparação ANTES vs APÓS escalonamento:")
comp = pd.DataFrame({
    'Média ANTES' : X[['idade', 'score_credito', 'log_saldo_conta']].mean().round(2),
    'Std ANTES' : X[['idade', 'score_credito', 'log_saldo_conta']].std().round(2),
    'Média APÓS' : X_scaled[['idade', 'score_credito', 'log_saldo_conta']].mean().round(4),
    'Std APÓS' : X_scaled[['idade', 'score_credito', 'log_saldo_conta']].std().round(4),
})
print(comp)
# Após StandardScaler: média ≈ 0 e desvio padrão ≈ 1 para todas as features

```

**Regra de ouro:** `.fit_transform()` só nos dados de **treino**. No conjunto de teste, use apenas `.transform()` com o scaler já ajustado — caso contrário, você “vaza” informação do teste para o treino (data leakage).

## ETAPA 8 — Seleção de Variáveis com Boruta

### O que é o Boruta?

O **Boruta** é um algoritmo de seleção de features baseado em Random Forest. Ele cria “cópias embaralhadas” (shadow features) de cada variável e compara a importância real com essas cópias aleatórias. Se uma variável não consegue bater a sua própria cópia aleatória, ela é rejeitada.

**Analogia:** O Boruta é como um concurso de dança onde cada candidato real compete contra um clone seu dançando aleatoriamente. Se você não consegue bater a si mesmo em modo aleatório, você está fora. Variáveis de ruído ( `ruído_1` , `ruído_2` , `ruído_3` ) são exatamente esses clones — e o Boruta vai descartá-las.

```

from boruta import BorutaPy
from sklearn.ensemble import RandomForestClassifier

# Split treino/teste antes do Boruta (importante!)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.20, random_state=42, stratify=y)

print(f"Treino: {X_train.shape[0]} amostras | Teste: {X_test.shape[0]} amostras")

# Modelo base do Boruta: Random Forest
rf_boruta = RandomForestClassifier(
    n_estimators=150,
    max_depth=7,
    n_jobs=-1,    # usa todos os núcleos disponíveis
    random_state=42
)

# Instanciar e treinar o Boruta
boruta_selector = BorutaPy(
    estimator=rf_boruta,
    n_estimators='auto',
    max_iter=80,    # iterações máximas
    alpha=0.05,    # nível de significância estatística
    verbose=1,
    random_state=42
)

boruta_selector.fit(X_train.values, y_train.values)
print("  Boruta concluído!")

```

## Interpretando os resultados

```

support    = boruta_selector.support_    # features confirmadas
support_weak = boruta_selector.support_weak_ # features tentativas

results_df = pd.DataFrame({
    'Feature' : X_train.columns,
    'Ranking' : boruta_selector.ranking_,
    'Confirmada': support,

```

```
'Tentativa' : support_weak & ~support
})

results_df['Status'] = 'Rejeitada '
results_df.loc[results_df['Confirmada'], 'Status'] = 'Confirmada '
results_df.loc[results_df['Tentativa'], 'Status'] = 'Tentativa '

results_df = results_df.sort_values('Ranking')
print(results_df[['Feature', 'Ranking', 'Status']])

print(f"\n Confirmadas : {support.sum()}")
print(f" Tentativas : {(support_weak & ~support).sum()}")
print(f" Rejeitadas : {(~support_weak).sum()}")
```

### O que cada status significa:

- **Confirmada:** variável estatisticamente relevante — manter
- **Tentativa:** evidência inconclusiva — decisão do analista
- **Rejeitada:** variável irrelevante (ex: ruído\_1 , ruído\_2 , ruído\_3 ) — descartar

---

## Pontos-Chave para Fixar

1. **Sempre trabalhe em uma cópia** ( `df = df_raw.copy()` ) — nunca altere o dataset original
2. A **estratégia de imputação depende da distribuição**: simétrica | média | assimétrica | mediana | categórica | moda | relacional | KNN
3. **Outliers não são sempre erros** — às vezes são os casos mais importantes (ex: fraude)
4. **Encoding errado distorce o modelo**: variável ordinal com One-Hot perde a ordem; variável nominal com Label Encoding cria uma ordem falsa
5. **drop\_first=True** no One-Hot evita multicolinearidade perfeita (Dummy Variable Trap)
6. **Escalone sempre depois do split** treino/teste — nunca antes (data leakage)
7. **Feature Engineering é onde o analista agrega mais valor** — conhecimento de domínio bate qualquer algoritmo automático

---

## Referências

- [Scikit-learn — Preprocessing](#)
- [Scikit-learn — Impute](#)
- [ydata-profiling](#) (antigo pandas-profiling)
- [Boruta-py](#)
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* — Cap. 2
- [Notebook da Aula 2 — `Aula2_PreProcessamento_ML.ipynb`] (material fornecido pelo professor)